

PLAYWRIGHT HANDBOOK

1. INTRODUCTION TO PLAYWRIGHT

★ What is Playwright ?

Playwright is an open-source framework by Microsoft for end-to-end testing of modern web applications.

It allows you to automate browsers, intercept network, handle multiple tabs, iframes, and much more.

★ Why Playwright ?

- ✓ Fast and reliable
- ✓ Cross-browser testing (Chromium, Firefox, WebKit)
- ✓ Auto-waiting and built-in assertions
- ✓ Rich set of APIs
- ✓ Supports Multiple Languages
- ✓ Powerful Debugging Tools
- ✓ Handles modern web apps easily (SPA, AJAX, Shadow DOM, etc.)

★ Key Features

- Cross-browser testing
- Auto-waiting
- Headless & Headed mode
- Parallel Execution
- Network Interception
- Screenshots, Videos, Traces
- Mobile Emulation
- Support for Multiple Languages

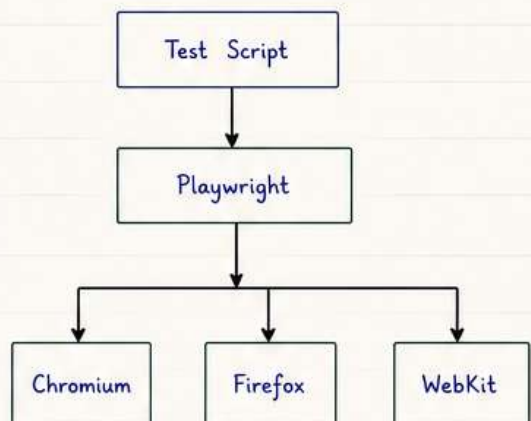
★ Supported Browsers

Chromium	Firefox	WebKit
(Chrome, Edge, Brave, etc.)	(Firefox Browser)	(Safari Browser)

★ Supported Languages

JavaScript	TypeScript	Python	.NET (C#)	Java
------------	------------	--------	-----------	------

★ Playwright Architecture



★ Installation

1. Using npm (Node.js)

```
npm init playwright@latest  
OR  
npm install @playwright/test
```

This will install Playwright and all required browsers (Chromium, Firefox, WebKit).

2. Install Browsers

```
npx playwright install
```

★ In Simple Words

Playwright acts as a bridge between your test code and browsers.

You write tests once and run them on multiple browsers reliably.

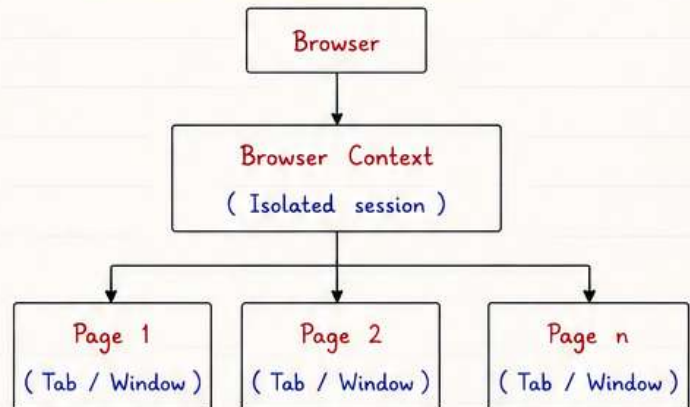
PLAYWRIGHT HANDBOOK

2. BROWSER & PAGE OPERATIONS

★ What is Browser, Context & Page ?

- **Browser** : The actual browser
(Chromium, Firefox, WebKit)
- **Context** : An isolated session in a browser. It has its own cookies, storage, cache, etc.
- **Page** : A single tab or window inside a context where we perform actions.

★ Browser → Context → Page



★ Launch Browser

```
const { chromium, firefox, webkit } = require('@playwright/test');

const browser = await chromium.launch({
  headless: false,    // true for headless mode
  slowMo: 50         // slow down (in ms)
});
```

★ Headless vs Headed

Headless (headless: true)	Headed (headless: false)
• Browser runs in background • Faster execution • No UI is displayed • Ideal for CI/CD	• Browser window is visible • Slower than headless • Good for debugging • Used in local execution

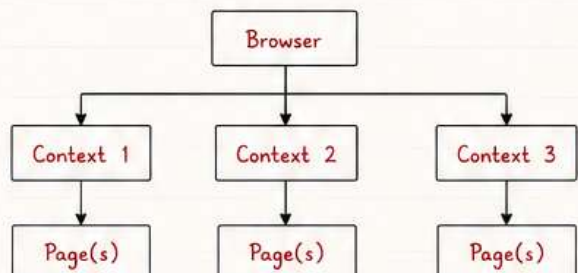
★ Create Context & Page

```
const context = await browser.newContext({
  viewport: { width: 1280, height: 720 }, // set viewport
  ignoreHTTPSErrors: true,
  baseURL: 'https://example.com'         // base url
});

const page = await context.newPage();    // open new tab
await page.goto('/login');               // navigate
```

★ Multiple Contexts

We can create multiple contexts in a single browser for different sessions.



★ Common Page Operations

- `await page.goto('https://example.com');` // navigate
- `await page.title();` // get page title
- `await page.url();` // get current url
- `await page.goBack();` // back
- `await page.goForward();` // forward
- `await page.reload();` // reload
- `await page.close();` // close current tab

★ Close Browser

```
await context.close(); // close context
await browser.close(); // close browser
```

★ Key Points

- Always create a new context for a new user/session.
- Context keeps session data isolated.
- Close context and browser after execution to free resources.

PLAYWRIGHT HANDBOOK

3. LOCATORS

★ What is a Locator ?

Locator is an object used to find and interact with elements on the page.

Playwright provides multiple ways to locate an element.

★ Common Locator Strategies

- **Role** : Based on ARIA role and name
- **Text** : Based on visible text
- **Label** : Based on label associated with input
- **Placeholder** : Based on placeholder text
- **Test Id** : Based on data-testid attribute
- **CSS** : Using CSS selectors
- **XPath** : Using XPath selectors

★ Locator Methods

Method	Example
locator()	page.locator('css=button')
getByRole()	page.getByRole('button', { name: 'Submit' })
getByText()	page.getByText('Login')
getByLabel()	page.getByLabel('Email')
getByPlaceholder()	page.getByPlaceholder('Enter email')
getByTestId()	page.getByTestId('user-name')

★ Locator Priority (Recommended Order)



★ Chaining Locators

We can narrow down element search using chaining.

```
// Example
const row = page.locator('table').locator('tr').nth(1);
const cell = row.locator('td').nth(2);
await cell.click();
```

★ Locator Operators

- **locator.first()** - first matching element
- **locator.last()** - last matching element
- **locator.nth(i)** - element at index i
- **locator.filter()** - filter locator by condition
- **locator.and()** - combine two locators
- **locator.or()** - match either locator

★ CSS vs XPath

CSS Selector	XPath
Faster	Slower
Easier to write	Complex syntax
Most preferred	Useful for complex DOM
Example: button.submit	//button[@class='submit']

★ Best Practices

- ✓ Use **getByRole()** whenever possible.
- ✓ Avoid CSS/XPath unless necessary.
- ✓ Use **data-testid** for stable automation.
- ✓ Avoid locating elements using index unless no other option.
- ✓ Keep locators unique and reliable.

PLAYWRIGHT HANDBOOK

4. USER ACTIONS

★ What are User Actions ?

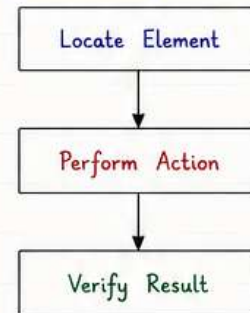
User Actions are the operations we perform on web elements to simulate real user behavior.

Playwright provides rich set of actions that are auto-wait enabled and reliable.

★ Common User Actions

- **Click** : Click on an element → `click()`
- **Double Click** : Double click on an element → `dblclick()`
- **Fill** : Enter text in input field → `fill()`
- **Clear** : Clear text from input field → `clear()`
- **Press Keys** : Press keyboard keys → `press()`
- **Check** : Check checkbox → `check()`
- **Uncheck** : Uncheck checkbox → `uncheck()`
- **Select Option** : Select from dropdown → `selectOption()`
- **Hover** : Hover over an element → `hover()`
- **Drag and Drop** : Drag element and drop → `dragTo()`
- **Upload File** : Upload file in input → `setInputFiles()`

★ Action Flow



★ Example : Fill and Click

```
await page.fill('#username', 'aman');  
await page.fill('#password', 'pass123');  
await page.click('button[type="submit"]');
```

★ Input Actions

Action	Description
<code>fill()</code>	Fills the element with text.
<code>type()</code>	Types like a user (one by one).
<code>clear()</code>	Clears the element value.
<code>press()</code>	Press a key or combination. (e.g. 'Enter')
<code>setInputFiles()</code>	Upload file to input[type="file"]

★ Select & Check Actions

Action	Description
<code>check()</code>	Checks a checkbox.
<code>uncheck()</code>	Unchecks a checkbox.
<code>selectOption()</code>	Select option in dropdown.
<code>isChecked()</code>	Verify checkbox is checked.
<code>isSelected()</code>	Verify option is selected.

★ Mouse Actions

- `hover()` : Hover over an element
- `click()` : Click on an element
- `dblclick()` : Double click
- `dragTo()` : Drag element and drop on target
- `moveTo()` : Move mouse to position
- `mouse.down()` : Press mouse button
- `mouse.up()` : Release mouse button

★ Key Points

- ✓ All actions are auto-wait enabled.
- ✓ Prefer locator based actions over element handles.
- ✓ Use meaningful actions to make tests readable and stable.

Drag and Drop Example

```
const source = page.locator('#drag');  
const target = page.locator('#drop');  
await source.dragTo(target);
```



PLAYWRIGHT HANDBOOK

5. ASSERTIONS & WAITING

★ What are Assertions ?

Assertions are used to verify the expected state of elements on the page.

Playwright uses `expect()` API for performing assertions.

★ Basic Assertion Syntax

```
import { expect } from '@playwright/test';

await expect(locator).toBeVisible();
```

↓ ↓
Locator Assertion

★ Common Assertions

Assertion	Description
<code>toBeVisible()</code>	Element is visible
<code>toBeHidden()</code>	Element is hidden
<code>toHaveText(text)</code>	Element has exact text
<code>toContainText(text)</code>	Element contains text
<code>toHaveValue(value)</code>	Input has expected value
<code>toHaveAttribute(name, value)</code>	Element has attribute value
<code>toBeEnabled()</code>	Element is enabled
<code>toBeDisabled()</code>	Element is disabled
<code>toHaveClass(className)</code>	Element has class
<code>toHaveURL(url)</code>	Page URL matches
<code>toHaveTitle(title)</code>	Page title matches

★ Auto Waiting (Built-in)

Playwright automatically waits for element to become ready before performing actions.

Auto-wait Conditions

- Element is attached to DOM
- Element is visible
- Element is stable
- Element is enabled
- Element is not receiving events

★ Explicit Waiting

We can use explicit waits when needed.

- `waitForSelector()`
- `waitForLoadState()`
- `waitForURL()`
- `waitForTimeout()`
- `waitForResponse()`
- `waitForFunction()`

★ Wait Methods

Method	Description	Example
<code>waitForSelector()</code>	Wait for element to appear in DOM	<code>await page.waitForSelector('#btn');</code>
<code>waitForLoadState()</code>	Wait for page load state	<code>await page.waitForLoadState('networkidle');</code>
<code>waitForURL()</code>	Wait for URL to match	<code>await page.waitForURL('**/dashboard');</code>
<code>waitForResponse()</code>	Wait for response matching URL/predicate	<code>await page.waitForResponse('**/api/data');</code>
<code>waitForTimeout()</code>	Wait for specified time (ms)	<code>await page.waitForTimeout(2000);</code>

★ Best Practices

- ✓ Prefer auto-wait over explicit wait.
- ✓ Use web-first assertions (`expect()`).
- ✓ Avoid `waitForTimeout()` unless absolutely necessary.
- ✓ Use specific locators to reduce flakiness.
- ✓ Wait for network responses instead of hard waits.

★ Example : Assertion with Waiting

```
await page.goto('https://example.com');
await page.waitForLoadState('networkidle'); // wait for page load
const title = page.locator('h1');
await expect(title).toBeVisible();           // assertion
await expect(title).toHaveText('Welcome');   // assertion
```

Wait for page
↓
Locate element
↓
Assert the expected state

PLAYWRIGHT HANDBOOK

6. FRAMES, WINDOWS & ALERTS

★ Frames (iframes)

Web pages may contain frames (iframes).

Playwright provides special locators

to work with them.

- `frameLocator()`
- `frame()`
- `contentFrame()`

★ Frame Example

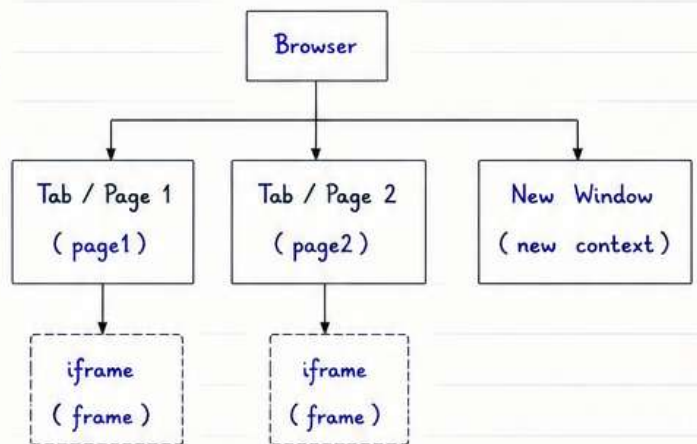
```
const frame = page.frameLocator('#my-frame');  
await frame.locator('input[name="username"]').fill('aman');  
await frame.locator('button[type="submit"]').click();
```

★ Multiple Windows / Tabs

- Playwright treats each tab/window as a Page.
- We can open new page and switch between them.

```
const page1 = await context.newPage();  
await page1.goto('https://example.com');  
const [page2] = await Promise.all([  
  context.waitForEvent('page'),  
  page1.click('a[target="_blank"]')  
]);  
await page2.goto('https://example.org');
```

★ Browser Window Structure



★ Alerts / Dialogs

Playwright can handle JavaScript dialogs.

- `alert()` : Information dialog
- `confirm()` : Confirmation dialog
- `prompt()` : Input dialog
- dialog event is used to handle them.

★ Alert / Dialog Example

```
page.on('dialog', async dialog => {  
  console.log(dialog.type()); // alert, confirm, prompt  
  console.log(dialog.message());  
  await dialog.accept(); // accept  
  // await dialog.dismiss(); // cancel  
});
```

★ Dialog Methods

Method	Description
<code>dialog.type()</code>	Returns type (alert, confirm, prompt)
<code>dialog.message()</code>	Returns message text
<code>dialog.accept([text])</code>	Accepts the dialog (For prompt, you can pass text)
<code>dialog.dismiss()</code>	Dismisses the dialog

★ Key Points

- ✓ Use `frameLocator()` to interact with elements inside iframes.
- ✓ Each tab/window is a separate Page.
- ✓ Use `context.waitForEvent('page')` to capture new tabs/windows.
- ✓ Always handle dialogs to avoid test failures.

★ In Simple Words

Frames allow you to work inside embedded pages.

Windows/Tabs help you handle multiple pages.

Alerts are pop-up messages that need to be accepted or dismissed.

PLAYWRIGHT HANDBOOK

7. API TESTING & NETWORK

★ Why API Testing in Playwright ?

- Playwright has powerful API testing capabilities using `APIRequestContext`.
- We can test backend APIs, seed data, authenticate and perform CRUD operations.
- Faster and independent of UI.

★ APIRequestContext

It is used to send HTTP requests.

Created using request fixture.

```
test('API test', async ({ request }) => {
  const res = await request.get('https://api.example.com/users');
  console.log(res.status());
});
```

★ HTTP Methods

Method	Description
GET	Retrieve data from server
POST	Create new resource
PUT	Update existing resource
PATCH	Partially update resource
DELETE	Delete resource

★ Request Options

- `headers` : Add custom headers
- `params` : Query parameters
- `data` : JSON body
- `timeout` : Set request timeout
- `response()` : Access response object

★ API Examples

```
// GET Request
const res1 = await request.get('/users');
expect(res1.status()).toBe(200);

// POST Request
const res2 = await request.post('/users', {
  data: { name: 'Aman', email: 'aman@test.com' }
});
expect(res2.status()).toBe(201);

// PUT Request
await request.put('/users/1', { data: { name: 'Aman Saini' } });

// DELETE Request
await request.delete('/users/1');
```

★ Network Monitoring

- We can listen to network events during test execution.
- Useful to verify API calls, status codes, payloads etc.



★ Listen to Requests

```
page.on('request', request => {
  console.log('Request URL:', request.url());
  console.log('Request Method:', request.method());
});
```

★ Listen to Responses

```
page.on('response', response => {
  console.log('Response URL:', response.url());
  console.log('Status:', response.status());
});
```

★ Route Interception (Mock / Modify)

- We can intercept and modify network requests.
- Useful for mock APIs, stubbing data etc.

```
await page.route('**/api/data', route => {
  route.fulfill({
    status: 200,
    contentType: 'application/json',
    body: JSON.stringify({ message: 'Success' })
  });
});
```

★ Key Points

- ✓ Use API testing to prepare data and validate backend.
- ✓ API tests are faster and more reliable.
- ✓ Use route interception to mock or modify APIs.
- ✓ Always assert status codes and response body.
- ✓ Combine API and UI tests for end-to-end coverage.

PLAYWRIGHT HANDBOOK

8. SCREENSHOTS, VIDEOS & TRACE VIEWER

Playwright provides powerful debugging and reporting tools to help us understand test failures easily.

★ Screenshots

- Capture screenshots of page or specific element.
- Helpful for debugging and in reports.

Method	Description
<code>page.screenshot()</code>	Full page screenshot
<code>locator.screenshot()</code>	Element screenshot
<code>fullPage: true</code>	Full scroll screenshot
<code>path</code>	Save screenshot to file

Example :

```
// Full page screenshot
await page.screenshot({ path: 'screens/home.png' });

// Element screenshot
await page.locator('#submit').screenshot({
  path: 'screens/submit.png'
});
```

★ Video Recording

- Record test execution as a video.
- Enabled in context options.

```
const context = await browser.newContext({
  recordVideo: {
    dir: 'videos/',
    size: { width: 1280, height: 720 }
  }
});
```

★ Trace Viewer

- Records everything (actions, network, DOM, console).
- Allows step-by-step debugging.
- Very helpful in identifying root cause.

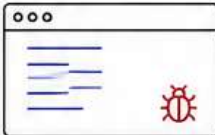
```
// Enable trace
await context.tracing.start({ screenshots: true,
  snapshots: true, sources: true });

// ... run your test ...

await context.tracing.stop({ path: 'trace.zip' });
```

★ Debugging Tools

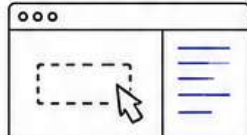
Debug Mode



Run tests in debug mode to pause on failures.

`npx playwright test --debug`

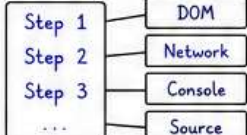
Inspector



Inspect elements and generate locators.

`npx playwright codegen`

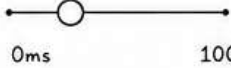
Trace Viewer



Open trace.zip in Trace Viewer to analyze.

`npx playwright show-trace trace.zip`

SlowMo



Slow down execution to see actions in slow motion.

`slowMo: 500`

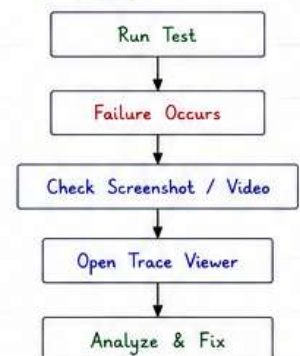
★ Common Screenshot Options

Option	Description
<code>fullPage</code>	Capture full scroll page
<code>clip</code>	Capture specific area
<code>type</code>	png / jpeg
<code>quality</code>	Image quality (0 - 100)
<code>animations: 'disabled'</code>	Disable animations
<code>caret: 'hide'</code>	Hide text caret
<code>scale: 'css'</code>	Use CSS pixels

★ When to Use What ?

Tool	Use Case
Screenshots	Quick visual proof
Videos	See the entire execution
Trace Viewer	Deep debugging
Inspector	Create accurate locators
Debug Mode	Pause and step through

★ Flow



★ Key Points

- ✓ Take screenshots for every important step.
- ✓ Enable video in CI for post execution review.
- ✓ Use trace viewer for detailed debugging.
- ✓ Use inspector to generate stable locators.
- ✓ Debug mode is very helpful during test authoring.

PLAYWRIGHT HANDBOOK

9. CONFIGURATION, REPORTS & PARALLEL EXECUTION

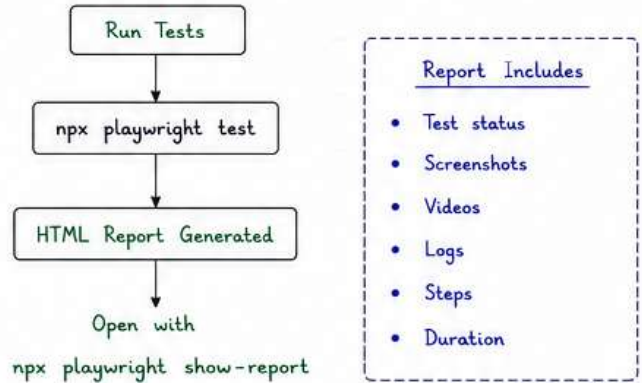
★ Configuration (playwright.config.ts)

All test settings are defined in config file.

```
import { defineConfig } from '@playwright/test';
export default defineConfig({
  testDir: './tests',
  timeout: 30*1000,
  expect: { timeout: 5000 },
  use: {
    headless: true,
    baseURL: 'https://example.com',
    viewport: { width: 1280, height: 720 },
    screenshot: 'only-on-failure',
  },
  projects: [
    { name: 'chromium', use: { browserName: 'chromium' } },
    { name: 'firefox', use: { browserName: 'firefox' } },
    { name: 'webkit', use: { browserName: 'webkit' } },
  ],
});
```

★ Reports

- Playwright generates HTML report.
- Easy to understand with screenshots, videos, logs and steps.



★ Parallel Execution

- Tests can run in parallel to save time.
- Use workers to control parallelism.



★ Workers Example

Workers	Description
1	Run tests one by one (default)
2	Run tests using 2 parallel workers
4	Run tests using 4 parallel workers
100%	Use all available CPU cores

★ Projects

- Run same tests on multiple browsers/devices.
- Configured in playwright.config.ts

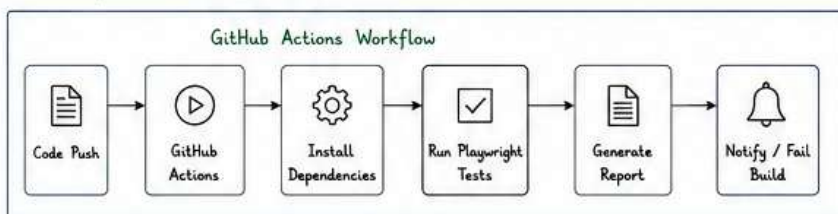


★ Useful CLI Commands

Command	Description
npx playwright test	Run all tests
npx playwright test tests/login.spec.ts	Run single test file
npx playwright test --debug	Run in debug mode
npx playwright test --headed	Run with browser UI
npx playwright show-report	Open HTML report
npx playwright codegen	Generate tests using Codegen
npx playwright install	Install browsers

★ CI/CD Integration

- Playwright integrates easily with CI/CD tools.
- Example with GitHub Actions.



```
- name: Install
  run: npm ci

- name: Run Playwright Tests
  run: npx playwright test

- name: Upload Report
  uses: actions/upload-artifact@v4
  with:
    name: playwright-report
    path: playwright-report/
```

★ Key Points

- ✓ Use config file for managing all settings.
- ✓ Run tests in parallel to save execution time.
- ✓ Always check HTML report and videos for failures.
- ✓ Use multiple projects for cross-browser testing.
- ✓ Integrate with CI/CD for automated test runs.

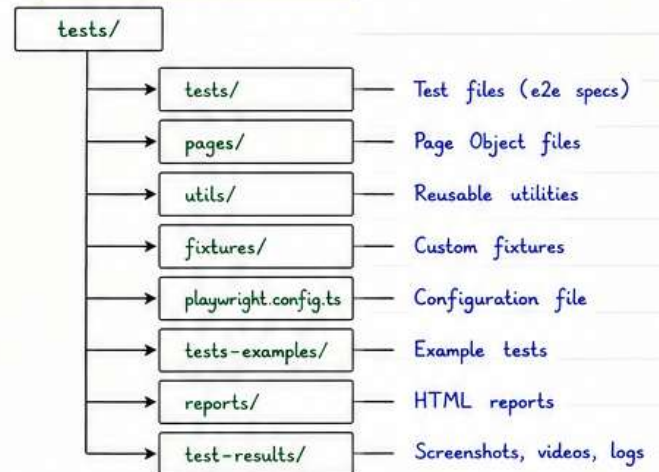
PLAYWRIGHT HANDBOOK

10. BEST PRACTICES, TIPS & CHEAT SHEET

★ Best Practices

- Use auto-wait and web-first assertions.
- Keep locators stable and reliable.
- Avoid hard waits (waitForTimeout).
- Use Page Object Model for large projects.
- Run tests in parallel for faster feedback.
- Keep tests independent and repeatable.
- Capture screenshots/videos on failure.
- Use meaningful test names and steps.

★ Folder Structure Example



★ Locator Strategies

Strategy	Example
By Role	<code>page.getByRole('button', { name: 'Submit' })</code>
By Text	<code>page.getByText('Submit')</code>
By Label	<code>page.getByLabel('Username')</code>
By Placeholder	<code>page.getByPlaceholder('Enter email')</code>
By Test Id	<code>page.getByTestId('user-input')</code>
By CSS	<code>page.locator('.btn-primary')</code>
By XPath	<code>page.locator('//input[@type="text"]')</code>

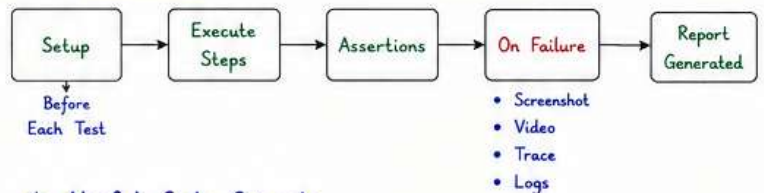
★ Assertions Cheat Sheet

Assertion	What it checks
<code>toBeVisible()</code>	Element is visible
<code>toBeHidden()</code>	Element is hidden
<code>toHaveText()</code>	Element has exact text
<code>toContainText()</code>	Element contains text
<code>toHaveValue()</code>	Input has expected value
<code>toHaveAttribute()</code>	Element has attribute value
<code>toBeEnabled()</code>	Element is enabled
<code>toBeDisabled()</code>	Element is disabled

★ Waits Guide

Wait	Purpose
Auto-wait (default)	Waits for element to be ready
<code>expect()</code> assertions	Retries until condition is met
<code>waitForSelector()</code>	Wait for element in DOM
<code>waitForLoadState()</code>	Wait for page load states
<code>waitForURL()</code>	Wait for URL to match
<code>waitForResponse()</code>	Wait for API response
<code>waitForFunction()</code>	Wait for custom condition

★ Test Flow



★ Command Cheat Sheet

Command	Use
<code>npx playwright test</code>	Run all tests
<code>npx playwright test <file></code>	Run a specific test file
<code>npx playwright test --ui</code>	Run tests with UI mode
<code>npx playwright test --debug</code>	Run in debug mode
<code>npx playwright show-report</code>	Open HTML report
<code>npx playwright codegen</code>	Generate locators
<code>npx playwright install</code>	Install browsers
<code>npx playwright test --project=chromium</code>	Run on specific browser

★ Useful Code Snippets

```
// Click and assert text
await page.getByRole('button', { name: 'Submit' }).click();
await expect(page.getByText('Success')).toBeVisible();
// Fill input
await page.getByLabel('Email').fill('aman@test.com');
// Select dropdown
await page.getByLabel('Country').selectOption('India');
// Upload file
await page.setInputFiles('input[type="file"]', 'file.pdf');
```

★ Quick Tips

- ✓ Prefer locators like `getByRole`, `getByLabel`, `getByText`.
- ✓ Keep tests small, focused and independent.
- ✓ Use fixture to share setup and data.
- ✓ Always review traces and videos on failure.
- ✓ Run tests on multiple browsers for better coverage.
- ✓ Use parallel execution for faster feedback.

★ In Simple Words

Playwright makes end-to-end testing reliable, fast and developer-friendly with auto-wait, powerful APIs, debugging tools and detailed reports.

- Aman Saini
Senior SDET

COMMENT PLAYWRIGHT

— TO GET —
COMPLETE PDF



Playwright

♥ SAVE | SHARE | FOLLOW ♥



aman_views



⇒ FOLLOW US FOR ⇒
UPCOMING FRAMEWORK NOTES